

Compiladores

Juliana da Costa Feitosa
juliana.feitosa@unesp.br
Docente UNESP - Bauru-SP

Aula 2

Fases de um compilador

Introdução

- Tratamos o compilador como uma caixa-preta que mapeia um **programa fonte** para um **programa objeto**
- Dentro dessa caixa existem duas partes:
 - **Análise**
 - **Síntese**

Análise

- Responsável por subdividir o **programa fonte** em partes constituintes e impõe uma **estrutura gramatical** sobre elas
- Essa estrutura é usada para criar uma representação intermediária do programa fonte
- A parte de análise também coleta informação sobre o programa fonte e armazena em uma estrutura chamada **tabela de símbolos**

Análise

- A parte de análise é conhecida como o **front-end** do compilador

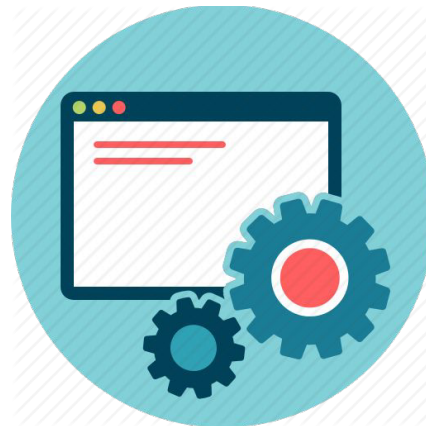


Síntese

- Responsável por construir o **programa objeto** desejado a partir da representação intermediária e das informações na **tabela de símbolos**

Síntese

- A parte de síntese é conhecida como o **back-end** do compilador



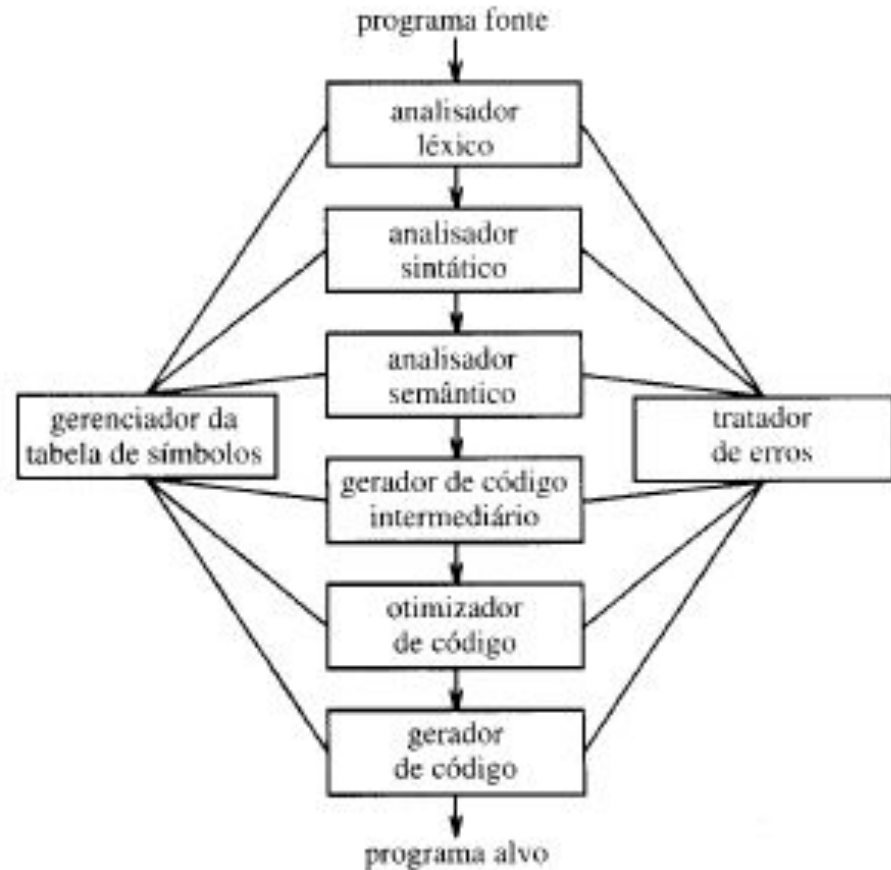
Análise e Síntese



Fases de um compilador

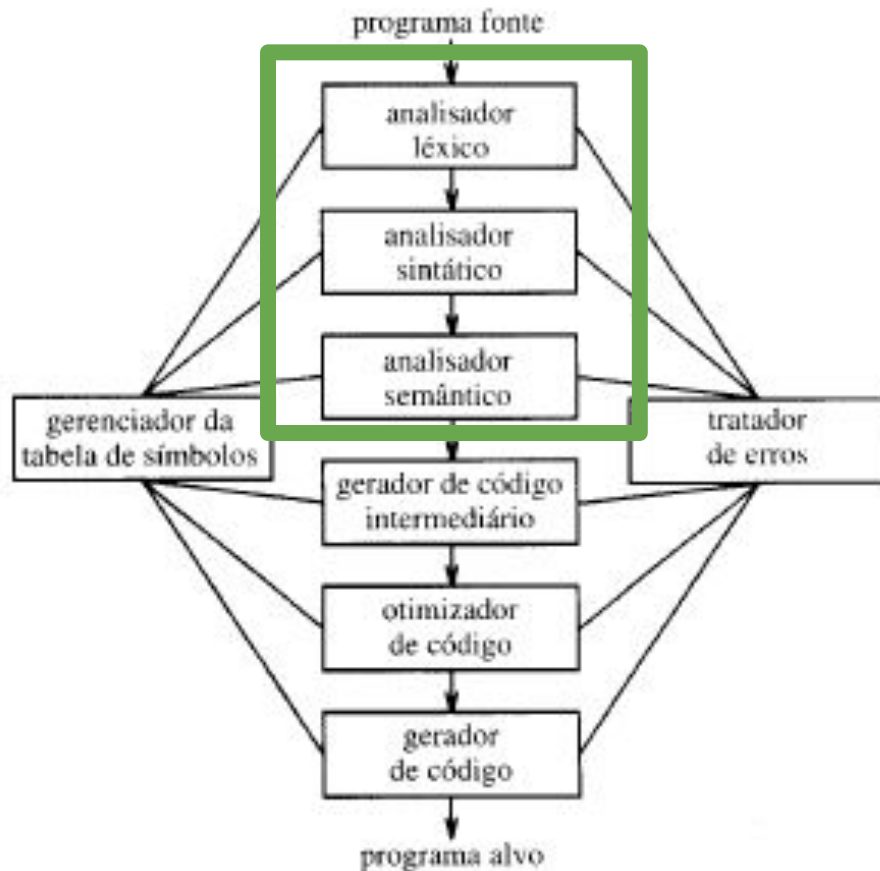
- Um compilador opera em fases, cada uma das quais transforma o **programa fonte de uma representação para outra**
- Na prática, algumas fases podem ser agrupadas

Fases de um compilador



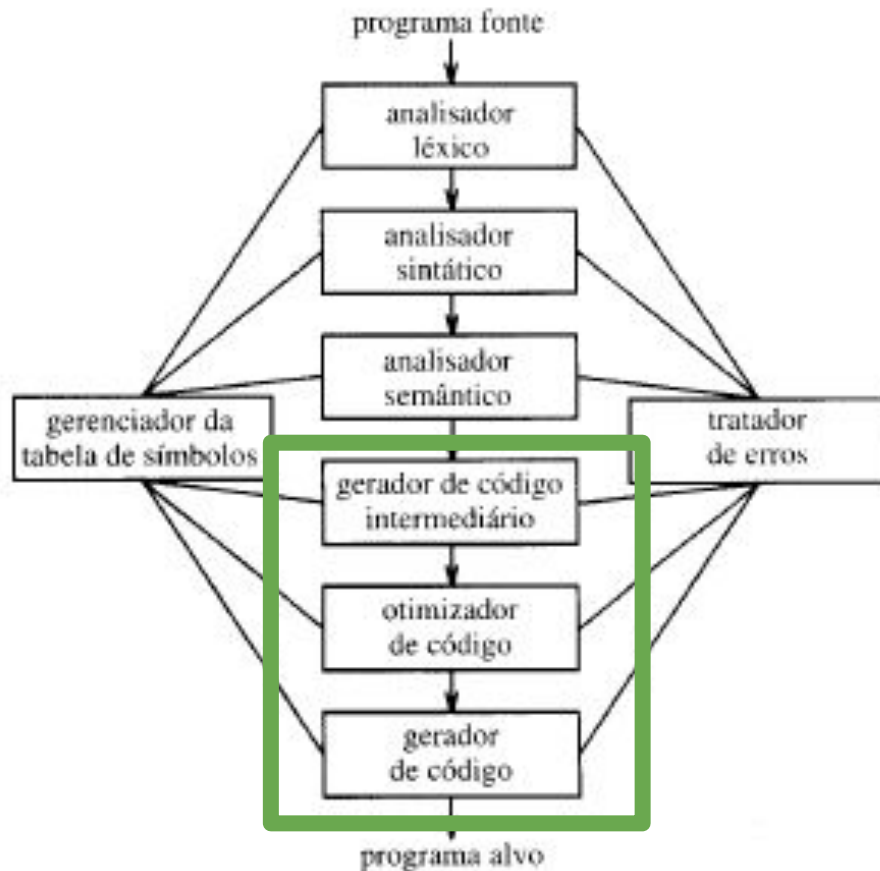
Fases de um compilador

- As três primeiras fases formam a parte de **análise** do compilador



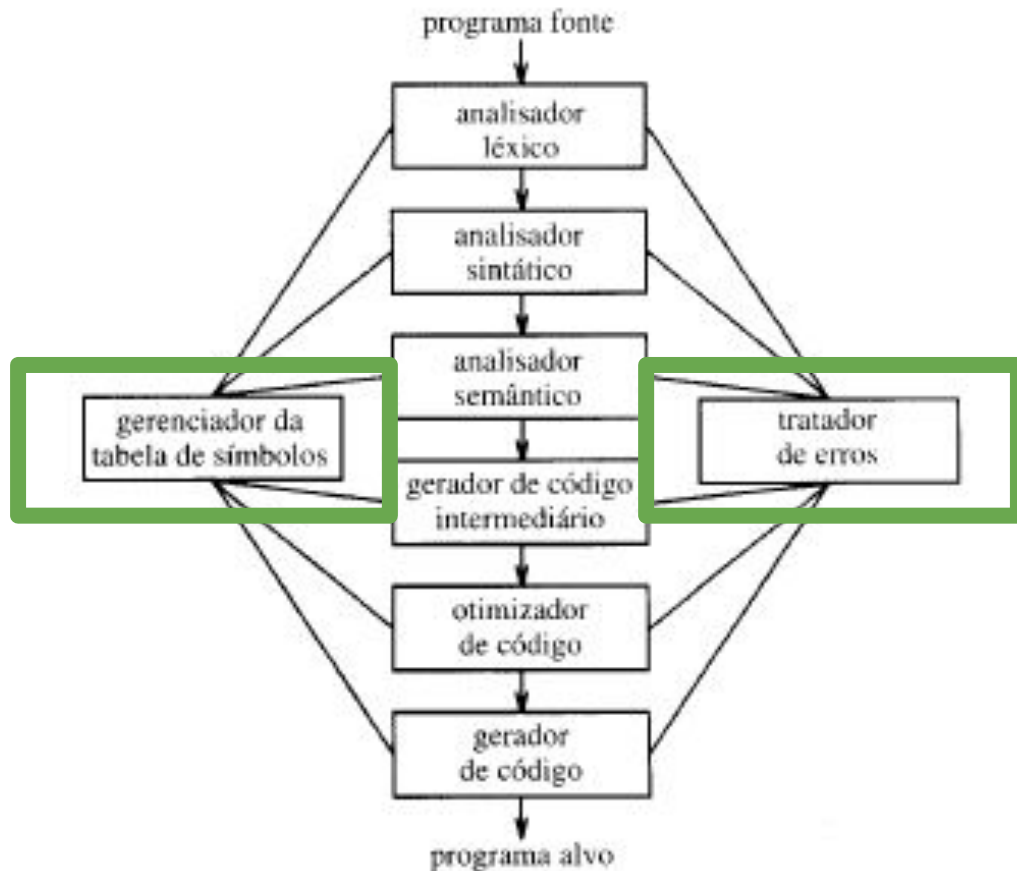
Fases de um compilador

- As três últimas fases formam a parte de **síntese** do compilador



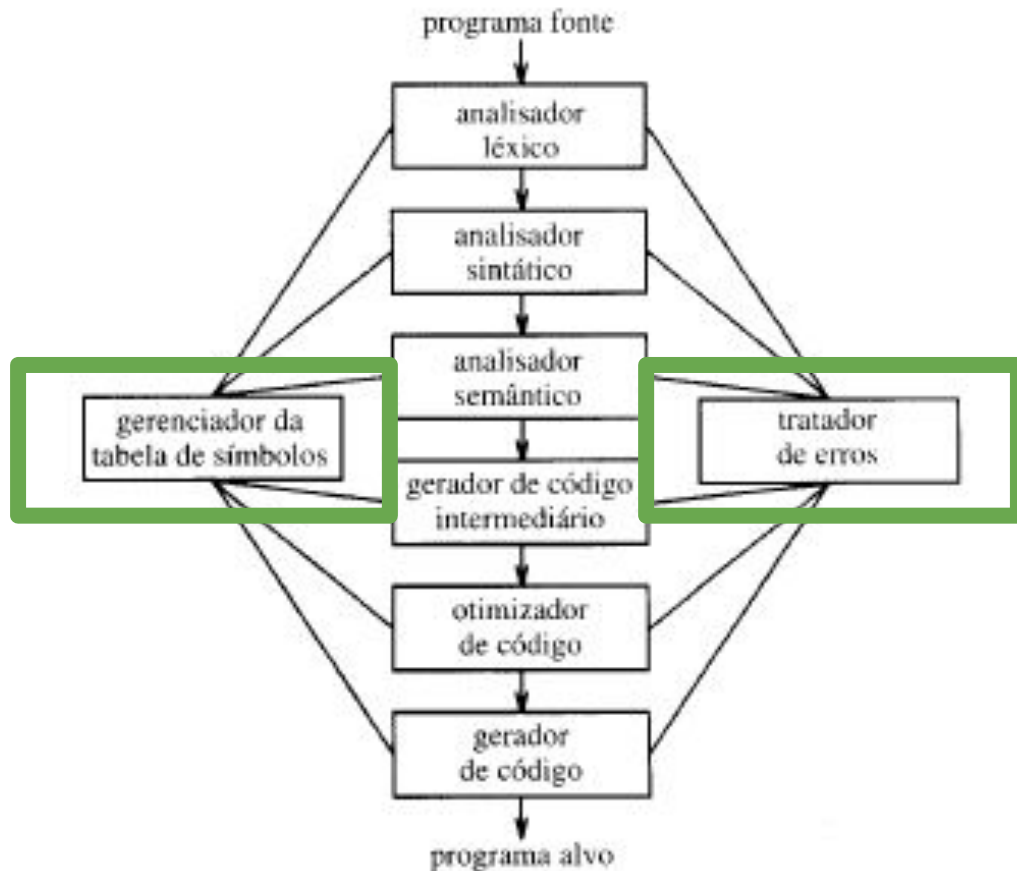
Fases de um compilador

- Também chamamos de fases o gerenciador da tabela de símbolos e o tratador de erros



Fases de um compilador

- São atividades que interagem com todas as demais



Análise Léxica

- A análise léxica tem como finalidade a separação e identificação dos elementos componentes do programa fonte; normalmente esses componentes são especificados através de expressões regulares

Análise Léxica

- Também pode ser chamada de **fase de leitura (scanning)**
- Lê o fluxo de caracteres que compõem o programa fonte e os agrupa em sequências significativas, chamadas **lexemas/átomo**
- Para cada lexema, o analisador léxico produz como saída um **token**

Análise Léxica

- Cada **token** representa uma sequência de caracteres de um programa fonte
- Pode ser uma palavra chave, um caractere, uma pontuação, ou conter vários caracteres

Análise Léxica

- Certos tokens são enriquecidos por um **valor léxico** ou **valor do atributo**

<nome-token, valor-atributo>

- **<Nome-token>**: é um símbolo abstrato que é usado durante a análise sintática (Ex: id)
- **<Valor-atributo>**: aponta para uma entrada na tabela de símbolos referente a esse token (Ex: 1, 2...)

Análise Léxica

- Exemplo

`montante := deposito_inicial + taxa_de_juros * 60`

TOKEN	LEXEMA
id1 ou id(montante)	montante
atribuicao ou :=	:=
id2 ou id(deposito_inicial)	deposito_inicial

Análise Léxica

- Exemplo

montante := deposito_inicial + taxa_de_juros * 60

TOKEN	LEXEMA
soma ou +	+
id3 ou id(taxa_de_juros)	taxa_de_juros
multiplicacao ou *	*
num(60) ou 60	60

Análise Léxica

- Exemplo

montante := deposito_inicial + taxa_de_juros * 60

LEXEMAS



id1 := id2 + id3 * 60

TOKENS

Análise Sintática

- A sintaxe de uma linguagem de programação deve descrever todos os aspectos relativos à forma de construção de programas corretos na linguagem.

Análise Sintática

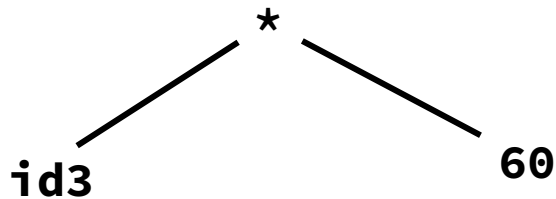
- O analisador sintático utiliza os primeiros componentes dos tokens produzidos pelo analisador léxico para criar uma representação intermediária, tipo árvore (**árvore de sintaxe**)
 - Cada interior representa uma **operação**
 - Os filhos dos nós representam **argumentos da operação**

Análise Sintática

- Exemplo

`id1 := id2 + id3 * 60`

A árvore possui um
nó interior rotulado
com `*`, com **id3** como
seu filho da esquerda
e **60** como filho da
direita

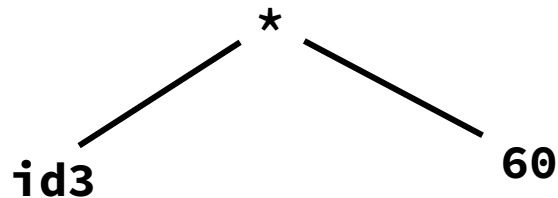


Análise Sintática

- Exemplo

id1 := id2 + id3 * 60

O nó * torna explícito
que devemos
multiplicar o valor de
id3 por **60**

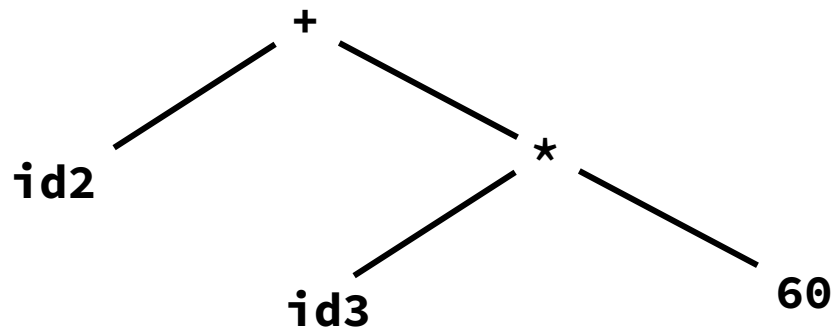


Análise Sintática

- Exemplo

id1 := id2 + id3 * 60

O nó **+** indica que devemos somar o resultado da multiplicação por **id2**

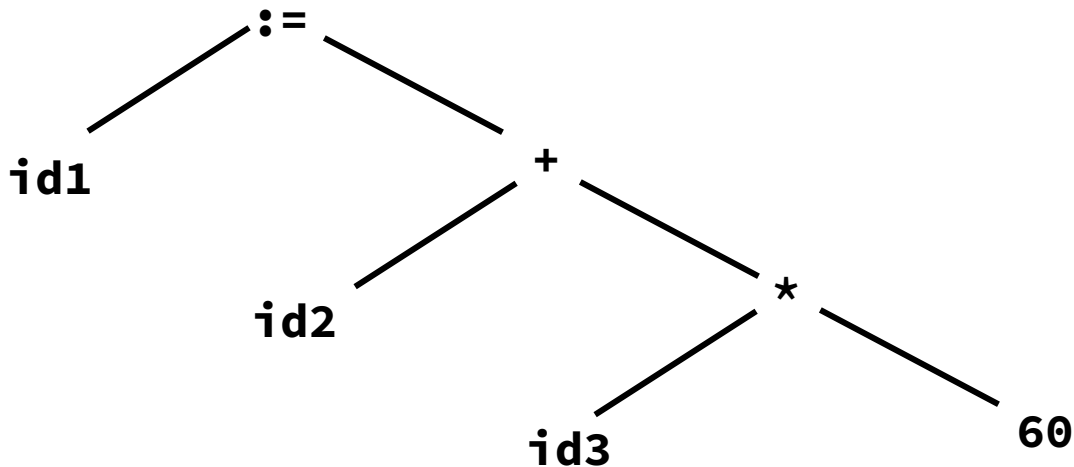


Análise Sintática

- Exemplo

id1 := id2 + id3 * 60

A raiz da árvore **:=**
indica que devemos
armazenar o
resultado dessa
adição em **id1**

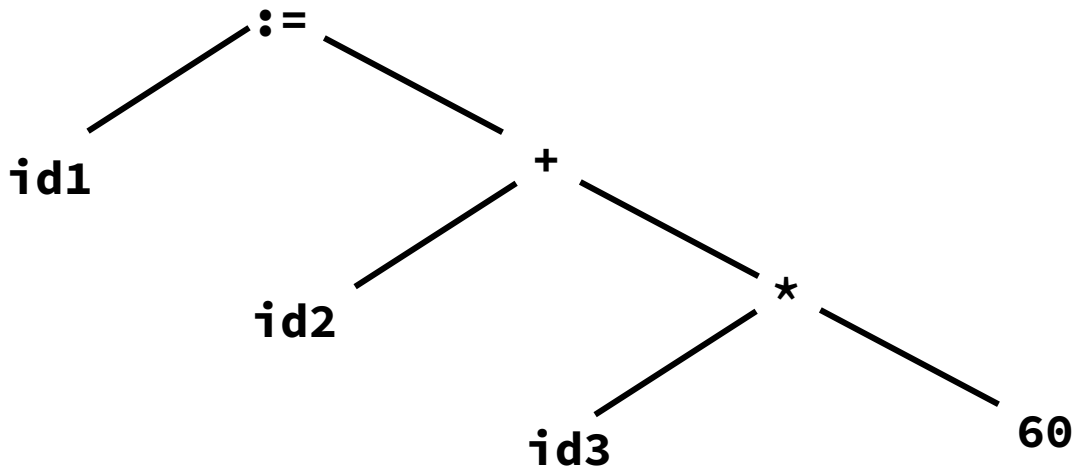


Análise Sintática

- Exemplo

id1 := id2 + id3 * 60

A ordem das operações é consistente com as **convenções normais de aritmética**



Análise Semântica

- Não existe um modelo matemático inteiramente adequado para descrever o que deve ser verificado na análise semântica, ao contrário do que acontece nas duas outras fases da análise, mais mecanismos como gramáticas de atributos têm sido usados com sucesso para simplificar a construção de analisadores semânticos.

Análise Semântica

- Utiliza a **árvore de sintaxe** e as **informações na tabela de símbolos** para verificar erros semânticos do programa fonte com a definição da linguagem
- Também reúne **informações sobre os tipos** e as salva na árvore de sintaxe ou na tabela de símbolos, para ser utilizado na geração de código intermediário

Análise Semântica

- **Verificação de tipo:** onde o compilador verifica se cada operador possui operandos compatíveis
- **Exemplo:**
 - Muitas linguagens de programação exigem que um **índice de um array seja um inteiro**
 - Por isso, o compilador precisa **informar um erro quando um real é utilizado como índice**

Análise Semântica

- No entanto, as especificações da linguagem pode permitir algumas **coerções (conversão)** de operadores
- Quando um operando aritmético binário é aplicado a um inteiro e a um real, **o compilador pode precisar converter o inteiro para real**

Análise Semântica

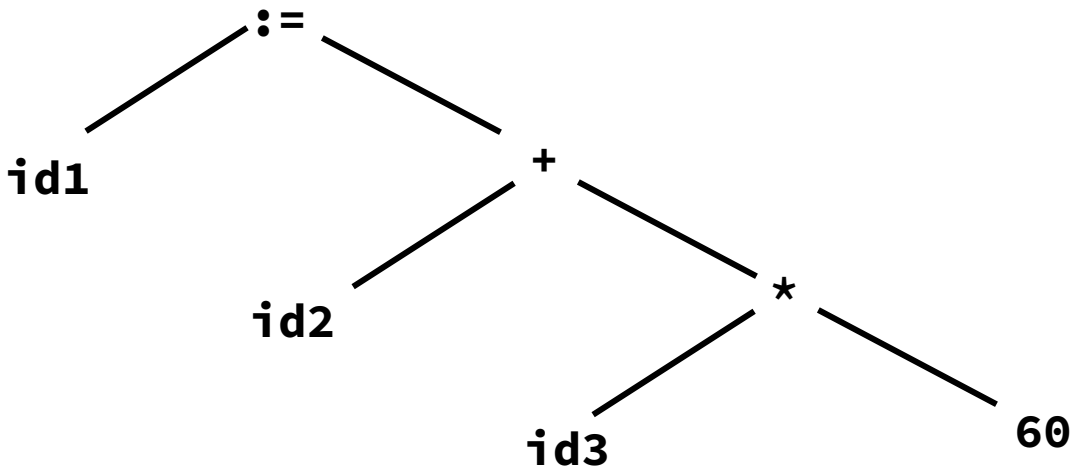
- Dentro de uma máquina, **um padrão de bits representando um inteiro, geralmente é diferente do padrão de bits para um real**
 - Mesmo que o número inteiro e o número real tenham o mesmo valor

Análise Semântica

- Exemplo

id1 := id2 + id3 * 60

Vamos supor que
id1, id2 e id3
tenham sido
declarados como
real e **60** seja um
inteiro

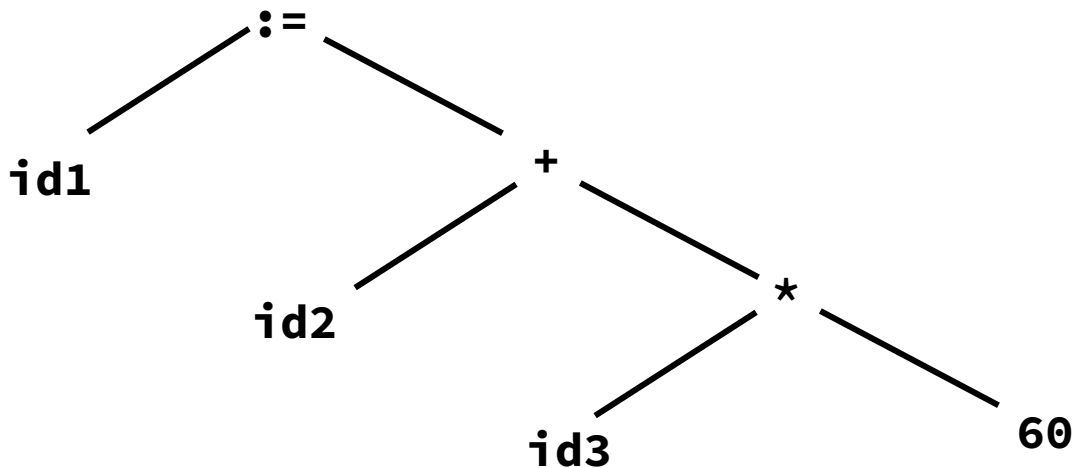


Análise Semântica

- Exemplo

id1 := id2 + id3 * 60

O enfoque geral é
o de **converter o**
inteiro em real

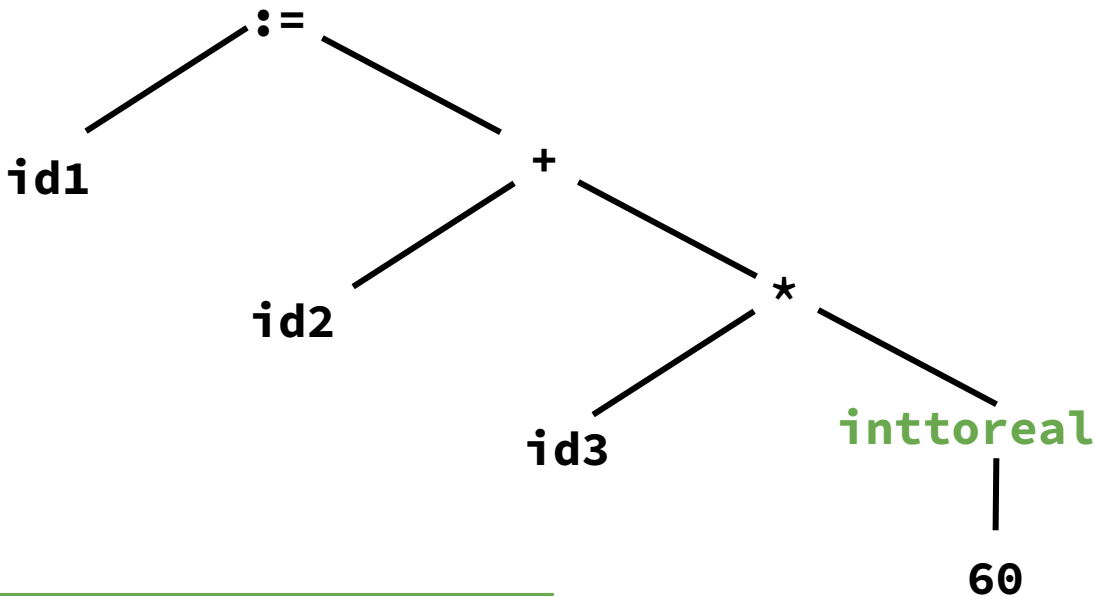


Análise Semântica

- Exemplo

id1 := id2 + id3 * 60

Para isso, é necessário a criação de um nó extra para o operador **inttoreal**, que converte explicitamente um **inteiro para real**



Geração de código intermediário

- No processo de traduzir um programa fonte para um código objeto, um **compilador pode produzir uma ou mais representações intermediárias**, as quais podem ter diversas formas
- O “front-end” e “back-end” se comunicam apenas através da representação intermediária. As **árvores de sintaxe** são um tipo dessa representação

Geração de código intermediário

- Após as análises sintáticas e semânticas, muitos compiladores geram uma representação intermediária explícita de **baixo nível ou do tipo de linguagem de montagem**. Essa representação deve ter duas propriedades importantes:
 - **ser facilmente produzida**
 - **ser facilmente traduzida**

Geração de código intermediário

- Considera-se uma forma intermediária chamada **código de três endereços**, que consiste em uma sequência de instruções do tipo assembly
- É uma linguagem de montagem para uma máquina

Geração de código intermediário

- Exemplo:

```
id1 := id2 + id3 * 60
```

Essas instruções determinam a **ordem** que essas operações devem ser realizadas

```
t1 = inttoreal(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

1º: *

2º: +

Geração de código intermediário

- Exemplo:

```
id1 := id2 + id3 * 60
```

Cada instrução de atribuição possui no máximo um operador do lado direito

```
t1 = inttoreal(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Geração de código intermediário

- Exemplo:

```
id1 := id2 + id3 * 60
```

O compilador precisa
gerar um **nome**
temporário para
guardar o valor
computado

```
t1 = inttoreal(60)  
t2 = id3 * t1  
t3 = id2 + t2  
id1 = t3
```

Geração de código intermediário

- Exemplo:

```
id1 := id2 + id3 * 60
```

Algumas instruções possuem menos de três operandos (no máximo são três)

```
t1 = inttoreal(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Geração de código intermediário

- Normalmente, as máquinas oferecem várias instruções (ou variantes de instruções) com características semelhantes, e o gerador deve escolher a mais apropriada entre elas

Otimização de código

- Essa fase realiza algumas transformações no código intermediário com o objetivo de produzir um código objeto melhor
 - **Melhor = mais rápido (código menor; código que consuma menos energia)**
- Uma boa estratégia é usar um algoritmo simples de geração de código intermediário seguido de otimização

Otimização de código

- Exemplo:

```
id1 := id2 + id3 * 60
```

O compilador pode deduzir que a conversão do 60 pode ser realizada uma vez (e nada mais), em tempo de compilação

```
t1 = inttoreal(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Otimização de código

- Exemplo:

```
id1 := id2 + id3 * 60
```

Assim, a operação
inttoreal pode ser
eliminada do código
substituindo-se o
inteiro 60 pelo real
60.0

```
t1 = inttoreal(60)  
t2 = id3 * 60.0  
t3 = id2 + t2  
id1 = t3
```

Otimização de código

- Exemplo:

```
id1 := id2 + id3 * 60
```

t3 é usado apenas uma vez na atribuição de seu valor para **id1**.

Assim, ele também pode ser eliminado

```
t1 = id3 * 60.0  
t3 = id2 + t2  
id1 = id2 + t1
```

Otimização de código

- Exemplo:

```
id1 := id2 + id3 * 60
```

Com isso, o código foi transformado em uma sequência de código menor

```
t1 = id3 * 60.0  
id1 = id2 + t1
```

Otimização de código

Neste outro exemplo, a otimização só funciona porque os valores de a, b, c e d não se alteram no trecho em questão.

Fonte	código intermediário original	código intermediário otimizado
<code>w := (a+b) + c;</code>	<code>t1 := a+b t2 := t1+c w := t2</code>	<code>t1 := a+b t2 := t1+c w := t2</code>
<code>x := (a+b) * d;</code>	<code>t3 := a+b t4 := t3*d x := t4</code>	<code>t4 := t1*d x := t4</code>
<code>y := (a+b) + c;</code>	<code>t5 := a+b t6 := t5+c y := t6</code>	<code>y := t2</code>
<code>z := (a+b) * d + e;</code>	<code>t7 := a+b t8 := t7*d t9 := t8+e z := t9</code>	<code>t9 := t4+e z := t9</code>

Otimização de código

- O número de otimizações de código realizadas por diferentes compiladores varia muito
- **Quanto mais otimizações, mais tempo é gasto nessa fase**
- Existem otimizações simples que melhoram de forma significativa o tempo de execução do programa objeto sem atrasar muito a compilação

Otimização de código

- Normalmente, as transformações realizadas no programa durante a otimização são simples: **eliminar ou alterar instruções**, ou ainda **mover instruções** para outras posições.
- A parte mais trabalhosa é a verificar que a transformação pode ser feita

Otimização de código

- As otimizações de um compilador precisam atender os **seguintes objetivos**:
 - Preservar a semântica do programa compilado
 - Melhorar o desempenho de muitos programas
 - Manter um tempo razoável de compilação
 - O esforço de engenharia empregado precisa ser administrável

Geração de código

- O gerador de código recebe como **entrada uma representação intermediária do programa fonte** e o **mapeia em uma linguagem objeto**
- **Na linguagem objeto**, é preciso selecionar os **registradores** (ou localização de memória) para cada variável utilizada

Geração de código

- Depois, os códigos intermediários são **traduzidos em sequências de instruções em linguagem de montagem que realizam a mesma tarefa**
- É preciso tomar cuidado com a **atribuição dos registradores**
- A organização de memória em tempo de execução depende da linguagem sendo compilada

Geração de código

- Exemplo:

id1 := id2 + id3 * 60

O primeiro e o segundo operandos de cada instrução especificam o **emissor e o receptor**, respectivamente

```
MOVF id3, R2  
MULF #60.0, R2  
MOVF id2, R1  
ADDF R2, R1  
MOVF R1, id1
```

Geração de código

- Exemplo:

`id1 := id2 + id3 * 60`

O **F** em cada uma das instruções indica a manipulação de valores reais

```
MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```

Geração de código

- Exemplo:

id1 := id2 + id3 * 60

O conteúdo de **id3** é carregado (MOVF) no registrador **R2**

```
MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```

Geração de código

- Exemplo:

id1 := id2 + id3 * 60

Depois, é
multiplicado (MULF)
pela constante **#60.0**

```
MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```

Geração de código

- Exemplo:

id1 := id2 + id3 * 60

significa que o valor
60.0 deve ser tratado
como uma constante
imediata

```
MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```

Geração de código

- Exemplo:

id1 := id2 + id3 * 60

id2 é movido para o
registrador **R1**

```
MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```

Geração de código

- Exemplo:

id1 := id2 + id3 * 60

Depois o valor do
registrador R1 é
somado com o valor
de R2

```
MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```

Geração de código

- Exemplo:

id1 := id2 + id3 * 60

Por fim, o valor de **R1**
é armazenado no
endereço de **id1**

```
MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```

Gerenciamento da tabela de símbolos

- Uma função essencial do compilador é **registrar os identificadores usados no programa fonte e coletar as informações sobre os seus diversos atributos**
- Esses atributos podem conter informações de **onde seu valor pode ser usado no programa**, como: espaço de memória alocado, tipo, escopo, etc.

Gerenciamento da tabela de símbolos

- A **tabela de símbolos** é uma estrutura de dados contendo um **registro para cada nome de variável, com campos para os atributos do nome**
- A estrutura de dados deve ser pensada para permitir que o compilador **encontre rapidamente o registro** para cada nome e **armazene ou recupere dados desse registro também rapidamente**

Gerenciamento da tabela de símbolos

- Ao detectar no programa fonte um identificador, o analisador léxico o instala na tabela de símbolos
- Entretanto, os atributos do identificador não podem ser automaticamente associados durante a análise léxica

Gerenciamento da tabela de símbolos

- Exemplo:

```
id1 := id2 + id3 * 60
```

Ao realizar a análise
semântica e a geração
de código
intermediário,
precisamos saber o tipo
dos identificadores

```
var id1, id2, id3: real;
```

Gerenciamento da tabela de símbolos

- Exemplo:

```
id1 := id2 + id3 * 60
```

O tipo real ainda **não**
será conhecido
quando os atributos
forem enxergados
pelo **analisador léxico**

```
var id1, id2, id3: real;
```

Gerenciamento da tabela de símbolos

- Por isso, as fases anteriores **colocam informações sobre os identificadores na tabela de símbolos** e em seguida **as usam de várias maneiras**, como por exemplo, **o tipo**

Detecção de erros

- **Do ponto de vista da teoria**, apenas programas corretos pertencem à linguagem, e os programas incorretos não tem nenhum interesse.
- Um programa ou é da linguagem (está correto) ou não é da linguagem (está incorreto).

Detecção de erros

- **Do ponto de vista da prática**, entretanto, no momento em que avisa que um programa está incorreto, um bom compilador deve ser capaz de indicar como esse fato foi descoberto, e, de alguma forma, ajudar o usuário a transformá-lo em um programa correto

Detecção de erros

- Cada fase pode encontrar erros e assim é preciso saber **tratá-los**, de forma que a compilação possa continuar, permitindo que seja detectado erros posteriores no programa fonte
- Um compilador que para no primeiro erro, não é tão útil

Detecção de erros

- As **fases de análise sintática e semântica** tratam usualmente de uma ampla fatia de erros detectáveis pelo compilador
- A **fase de análise léxica** pode detectá-los quando os caracteres remanescentes na entrada não formem qualquer **token de linguagem**

montante := depósito_inicial + taxa_de_juros * 60

Tabola de Símbolos

1	montante	...
2	depósito_inicial	...
3	taxa_de_juros	...
4		

analizador léxico

$id_1 := id_2 + id_3 * 60$

analizador sintático

$$\begin{array}{c}
 id_1 \\
 \swarrow \quad \searrow \\
 := \quad + \quad * \quad 60 \\
 \quad \swarrow \quad \searrow \quad \swarrow \\
 \quad id_2 \quad id_3
 \end{array}$$

analizador semântico

$$\begin{array}{c}
 id_1 \\
 \swarrow \quad \searrow \\
 := \quad + \quad * \quad \text{inttoreal} \\
 \quad \swarrow \quad \searrow \quad \swarrow \quad \downarrow \\
 \quad id_2 \quad id_3 \quad 60
 \end{array}$$

gerador código intermediário

$$\begin{array}{l}
 temp1 := \text{inttoreal}(60) \\
 temp2 := id3 * temp1 \\
 temp3 := id2 + temp2 \\
 id1 := temp3
 \end{array}$$

otimizador de código

$$\begin{array}{l}
 temp1 := id3 * 60.0 \\
 id1 := id2 + temp1
 \end{array}$$

gerador de código

$$\begin{array}{l}
 \text{MOV}F \ id3, R2 \\
 \text{MUL}F \ #60.0, R2 \\
 \text{MOV}F \ id2, R1 \\
 \text{ADD}F \ R2, R1 \\
 \text{MOV}F \ R1, id1
 \end{array}$$

Passos da compilação

- As fases do compilador podem ser executadas em **sequência ou ter a sua execução combinada**
- Dessa forma, podemos ter dois tipos de compilação:
 - **Em vários passos**
 - **Em um único passo**

Passos da compilação

- **Vários passos**
 - A execução de algumas fases terminam antes de iniciar a execução das fases seguintes
- **Em um único passo**
 - O programa objeto é produzido na medida em que o programa fonte é processado

Passos da compilação

- Normalmente o compilador de um único passo é mais eficiente
- É mais fácil introduzir **as fases de otimização num compilador de vários passos**

Impacto dos Compiladores

- Os avanços nas linguagens de programação impõem **novas demandas** sobre os projetistas de compiladores
 - É preciso criar algoritmos e representações para **traduzir e dar suporte** aos novos recursos das linguagens

Impacto dos Compiladores

- Os compiladores também são responsáveis por **efetivar** o uso das arquiteturas de computador de alto desempenho nas aplicações dos usuários
 - Os compiladores são usados como uma **ferramenta na avaliação dos conceitos arquitetônicos** antes que um computador seja montado

Tradução de programa

- Os compiladores podem ser aplicados para a tradução de diferentes tipos de linguagens
 - **Tradução binária:** tradução do código binário de uma máquina para o de outro, permitindo que uma máquina execute programas compilados originalmente para outro conjunto de instruções

Tradução de programa

- Os compiladores podem ser aplicados para a tradução de diferentes tipos de linguagens
 - **Síntese de hardware:** traduzem automaticamente descrições em RTL (Register Transfer Level) para portas, que são mapeadas para transistores e eventualmente para um layout físico

Tradução de programa

- Os compiladores podem ser aplicados para a tradução de diferentes tipos de linguagens
 - **Interpretadores de consulta de Banco de Dados:** compilação de comandos que consultam registros de uma BD conforme os operadores relacionais e booleanos da linguagem de consulta (e.g. SQL)

Tradução de programa

- Os compiladores podem ser aplicados para a tradução de diferentes tipos de linguagens
 - **Simulação compilada:** compilar um projeto para produzir código de máquina que simula esse mesmo projeto em particular novamente